

**INSTITUTO FEDERAL DE EDUCAÇÃO CIÊNCIA E TECNOLOGIA DE SÃO
PAULO**

**CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS INTEGRADO AO
ENSINO MÉDIO**



SABOR E CLIC

Documentação do Projeto Integrador do Terceiro Bimestre

São Paulo

2026

SUMÁRIO

- 1 Apresentação do projeto
- 2 Objetivos e escopo
- 3 Concepção do sistema e integração entre disciplinas
- 4 Atores e responsabilidades
- 5 Fluxo geral do produto
- 6 Reserva
- 7 Sessão
- 8 Cardápio e carrinho
- 9 Pedidos
- 10 KDS
- 11 Administração de Bancadas e catálogo
- 12 Arquitetura da aplicação
- 13 Lógica JavaScript
- 14 Arquitetura Flask e rotas essenciais
- 15 Banco de dados e DER
- 16 Segurança e níveis de acesso
- 17 Identidade visual e prototipação
- 18 Requisitos funcionais e não funcionais
- 19 Tecnologias utilizadas
- 20 O que concluímos e próximas etapas
- 21 Atendimento aos requisitos do terceiro bimestre
- 22 Estado atual da implementação e integração
- Referências

1 APRESENTAÇÃO DO PROJETO

Nós desenvolvemos o Sabor e Clic como uma proposta de sistema para organizar a operação de uma cozinha compartilhada com Bancadas reserváveis e atendimento gastronômico integrado. Nosso ponto de partida foi um problema recorrente: quando Reservas, disponibilidade de espaços, Cardápio e Pedidos são controlados por planilhas ou anotações isoladas, surgem conflitos de horário, falta de rastreabilidade e dificuldade para acompanhar a produção. Por isso, concebemos uma experiência única que liga o Cliente à cozinha e à administração.

Nossa documentação acompanha o fluxo do produto: Reserva → Sessão → Cardápio e Pedidos → KDS. Escolhemos esse encadeamento porque cada Pedido precisa existir dentro de um contexto de atendimento. A Reserva define quem utilizará qual Bancada e em qual período; a Sessão representa o uso efetivo dessa Reserva; o Cardápio permite selecionar itens; o Pedido registra a solicitação; e o KDS oferece à cozinha uma visão organizada da produção.

Ao longo do terceiro bimestre, reunimos contribuições de Artes Visuais e Design, JavaScript, Python, Banco de Dados e Segurança. Nosso protótipo estático demonstra as principais telas e interações. A modelagem de dados descreve as entidades e os relacionamentos necessários para sustentar a aplicação. Também implementamos uma API Flask executável com dados simulados em memória e rotas JSON para o fluxo principal.

2 OBJETIVOS E ESCOPO

2.1 Objetivo geral

Nosso objetivo é construir um sistema web que permita administrar Bancadas gastronômicas, criar Reservas, iniciar Sessões, consultar o Cardápio, enviar Pedidos e acompanhar seu preparo no KDS. Pretendemos centralizar dados que normalmente estariam dispersos e oferecer uma visão adequada para Cliente, Cozinheiro e Administrador.

2.2 Objetivos específicos

- Representar o fluxo completo do atendimento de forma clara e navegável.
- Modelar as entidades, atributos e relacionamentos em um DER e em um dicionário de dados.
- Aplicar JavaScript moderno para manipular o DOM, manter o carrinho e calcular valores em memória.
- Definir uma API Flask organizada por responsabilidades, com comunicação por JSON.
- Planejar autenticação, validação e RBAC para proteger ações e dados.
- Criar identidade visual coerente e interfaces para Cardápio Digital e KDS.

2.3 Limites desta etapa

Nesta etapa concluímos o protótipo estático, a documentação conceitual de dados e uma API Flask executável com dados mantidos em memória. O front-end ainda não utiliza Fetch e não está integrado à API. Banco físico, autenticação real, cookies de login, RBAC no servidor, POO completa em JavaScript e persistência transacional permanecem para o quarto bimestre.

3 CONCEPÇÃO DO SISTEMA E INTEGRAÇÃO ENTRE DISCIPLINAS

Nós organizamos o projeto como um sistema interdisciplinar. Design define como o usuário compreende estados e ações; JavaScript transforma a interface em uma experiência dinâmica; Flask expõe contratos JSON e aplica as validações desta etapa; Banco de Dados descreve a persistência futura por meio dos modelos conceituais; e Segurança define identidade, permissão e regras de entrada. A qualidade do fluxo depende da concordância entre essas partes.

Disciplina	Contribuição realizada	Evolução arquitetural
Design	Paleta, tipografia, componentes, telas por perfil, Cardápio e KDS.	Fidelidade ao protótipo, acessibilidade e testes de usabilidade.
JavaScript	JavaScript modularizado por responsabilidade, DOM, carrinho, validações, cálculos em memória e perfis simulados.	Classes ES6+, Fetch/JSON, async/await e integração real com Flask no quarto bimestre.
Python e Flask	Factory Flask, Blueprints, Routes, Controllers, Models em memória e endpoints JSON para Reservas, Sessões, Cardápio, Pedidos e KDS.	Persistência relacional, autenticação/RBAC, integração Fetch e services adicionais quando necessários.
Banco de Dados	DER, MER, EER e dicionário de dados.	Modelo físico, scripts SQL, consultas e triggers.
Segurança	Definição de perfis e separação visual.	Autenticação, cookies de login, RBAC no servidor e sanitização.

4 ATORES E RESPONSABILIDADES

Ator	Objetivo no sistema	Principais ações
Cliente	Utilizar uma Bancada e consumir durante uma Sessão.	Consultar Bancadas, reservar, abrir Sessão, montar carrinho, enviar e acompanhar Pedidos.
Cozinheiro	Organizar a produção.	Consultar fila, iniciar preparo, marcar Pedido pronto e entregue, acompanhar tempos.
Administrador	Configurar e supervisionar a operação.	Gerenciar Bancadas, Reservas, Pratos, Ingredientes, usuários e relatórios.

Nós representamos os três atores por menus distintos. Essa separação melhora a compreensão do protótipo, mas sabemos que visibilidade de menu não equivale a autorização. Na aplicação integrada, cada requisição protegida precisará ser validada pelo Flask de acordo com o usuário autenticado e seu papel.

5 FLUXO GERAL DO PRODUTO

O fluxo começa quando o Cliente consulta uma data e um intervalo. O sistema compara capacidade e disponibilidade das Bancadas. Com uma opção válida, cria a Reserva e preserva o período e o valor acordado. No início do atendimento, a Reserva gera uma Sessão única. É a Sessão ativa que autoriza o acesso ao Cardápio e a criação de Pedidos. A cozinha recebe o Pedido no KDS, altera seu estado e devolve esse progresso ao Cliente. No encerramento, a Sessão deixa de aceitar novos Pedidos e conserva o histórico.

Etapa	Estado de entrada	Processamento	Saída
Reserva	Bancada disponível.	Validamos período, capacidade e conflito.	Reserva confirmada.
Sessão	Reserva válida.	Criamos contexto único e ativamos o atendimento.	Sessão ativa.
Cardápio	Sessão ativa.	Exibimos Pratos disponíveis e mantemos carrinho.	Itens selecionados.
Pedido	Carrinho válido.	Revalidamos itens, preços e quantidades.	Pedido recebido.
KDS	Pedido recebido.	Organizamos fila e controlamos transições.	Pedido em preparo, pronto e entregue.

Esse fluxo também orienta a estrutura de dados. O Pedido não precisa repetir Cliente ou Bancada, porque sua Sessão conduz à Reserva e a Reserva identifica ambos. Essa decisão reduz redundância e mantém uma única fonte para o contexto do atendimento.

6 RESERVA

6.1 Lógica de negócio

Nós tratamos a Reserva como o compromisso de uso de uma Bancada durante um intervalo. Para criar uma Reserva, precisamos identificar o Cliente, a Bancada, o início, o fim, o número de pessoas e o valor. A Bancada deve estar ativa, comportar o grupo e não possuir outra Reserva incompatível. O fim precisa ser posterior ao início.

Há sobreposição quando o início solicitado ocorre antes do fim existente e o fim solicitado ocorre depois do início existente. A API em memória já executa essa verificação antes de criar a Reserva. No quarto bimestre, a mesma regra deverá ocorrer dentro da transação do banco para evitar conflitos entre solicitações simultâneas.

6.2 Contribuição de cada disciplina

Em Design, criamos cartões de Bancada, etiquetas de disponibilidade, filtros e formulário de Reserva. Em JavaScript, utilizamos o array benches e templates para renderizar capacidade, recursos, preço e estado. Em Banco de Dados, relacionamos CLIENTE e BANCADA à RESERVA. Em Flask, implementamos consulta de Bancadas, criação e consulta de Reservas com validações em memória. Autorização por proprietário e perfil permanece planejada para o quarto bimestre.

```
const benches = [  
  {id:1, nome:"Bancada Toscana", cap:4, preco:85,  
    status:"Disponível", recursos:["Cooktop","Pia","Tomadas"]}  
];
```

Trecho 1 - Estrutura real utilizada para representar Bancadas no protótipo.

No navegador, os dados permanecem em memória e os botões simulam confirmação. Separadamente, a API Flask já recebe JSON, valida capacidade, datas e conflitos, e devolve respostas JSON com códigos 201, 400, 404 ou 409. A ligação dessas duas partes por Fetch fica para o quarto bimestre.

7 SESSÃO

Nós adotamos Sessão como entidade central do atendimento. Uma Reserva descreve o período contratado; a Sessão registra o uso efetivo e reúne os Pedidos daquele período. O model em memória garante uma Sessão por Reserva e registra identificador, código de acesso, estado e datas de criação, início e encerramento.

Nesta implementação, a Sessão é criada como ativa e pode ser encerrada por endpoint próprio. Somente uma Sessão ativa aceita Pedidos, e o Flask verifica esse estado em cada criação. Autorização por usuário e validade temporal serão acrescentadas com a autenticação real.

Na tela que desenvolvemos, mostramos tempo restante, quantidade de Pedidos, consumo, pessoas, Pedidos vinculados e ações de navegação. Em JavaScript, essa tela é uma função do objeto views. No backend, SESSAO referencia uma RESERVA e origina vários PEDIDOS por id_sessao. No banco futuro, essa relação será protegida por chaves e unicidade.

8 CARDÁPIO E CARRINHO

Nosso Cardápio Digital apresenta foto, nome, descrição, categoria, preço e disponibilidade. Criamos oito produtos demonstrativos e organizamos a experiência em Entradas, Pratos, Bebidas e Sobremesas. A interface desabilita itens indisponíveis e a API repete essa validação ao criar um Pedido, usando o catálogo mantido em memória.

8.1 Lógica do carrinho

```
function addCart(id) {  
  const p=products.find(x=>x.id===id);  
  if(p.status!=="Disponível") return;  
  const ex=state.cart.find(x=>x.id===id);  
  if(ex) ex.qtd++;  
  else state.cart.push({id:p.id,nome:p.nome,preco:p.preco,qtd:1});  
}  
function cartTotal() {  
  return state.cart.reduce((s,i)=>s+i.preco*i.qtd,0);  
}
```

Trecho 2 - Adição de item e cálculo do total no protótipo.

Primeiro localizamos o produto pelo identificador. Se ele estiver indisponível, encerramos a função. Depois procuramos o mesmo item no carrinho: quando existe, incrementamos a quantidade; quando não existe, adicionamos um novo objeto. O total é calculado com reduce, acumulando preço multiplicado pela quantidade. Essa lógica satisfaz o objetivo do terceiro bimestre de manipular arrays, objetos e cálculos em memória.

A API recebe apenas identificadores e quantidades, consulta os preços atuais do catálogo em memória, valida cada Prato e calcula os subtotais e o total. O front-end ainda não envia esses dados por Fetch; essa integração permanece para o quarto bimestre.

9 PEDIDOS

O Pedido agrupa itens solicitados durante uma Sessão. Definimos que ele deve possuir ao menos um ITEM_PEDIDO, estado, valor total, tempo estimado e data de criação. ITEM_PEDIDO preserva quantidade, preço unitário praticado e subtotal. Mesmo que o preço do catálogo mude posteriormente, o histórico continua representando a compra original.

Ao confirmar um Pedido nesta etapa, o servidor verifica a existência e o estado ativo da Sessão, a lista não vazia, as quantidades positivas e a disponibilidade dos Pratos. Em seguida, busca os preços, calcula subtotais e devolve JSON. Autenticação, propriedade da Sessão e transação em banco serão implementadas no quarto bimestre.

Estado	Significado	Próxima ação esperada
AGUARDANDO ou RECEBIDO	Pedido aceito e visível para a cozinha.	Iniciar preparo ou cancelar conforme regra.
EM PREPARO	Produção iniciada.	Marcar como pronto.
PRONTO	Pedido finalizado pela cozinha.	Marcar como entregue.
ENTREGUE	Ciclo operacional concluído.	Somente consulta histórica.
CANCELADO	Pedido interrompido.	Somente consulta ou procedimento administrativo.

10 KDS

O Kitchen Display System é o painel do Cozinheiro. Nós o organizamos em colunas para tornar o estado imediatamente visível: Recebidos, Em preparo e Prontos. Cada ticket apresenta número, Bancada, tempo e itens. Cores e etiquetas reforçam prioridade e atraso. Essa estrutura reduz a necessidade de abrir cada Pedido separadamente.

No protótipo, os tickets continuam sendo exemplos visuais. A API Flask já oferece consultas JSON para a fila do KDS e inclui o contexto de Sessão, Reserva e Bancada de cada Pedido. A interface ainda não consome essa fila. No quarto bimestre, o KDS usará Fetch e as mudanças de estado serão persistidas e validadas pelo servidor.

Design contribui com hierarquia, cores e leitura rápida. JavaScript controla a renderização demonstrativa sem F5. Flask monta a fila em memória a partir dos Pedidos e de seus vínculos. Banco registrará estados e horários no quarto bimestre, e Segurança limitará as ações a Cozinheiro e Administrador quando a autenticação real for implementada.

11 ADMINISTRAÇÃO DE BANCADAS E CATÁLOGO

O Administrador configura os recursos utilizados nos fluxos anteriores. Desenvolvemos telas para Bancadas, Reservas, Produtos, Ingredientes, substituições, usuários e relatórios. As operações exibidas ainda são demonstrativas, mas definem a experiência e os dados necessários para os futuros CRUDs.

A exclusão de Prato deve ser lógica por meio do atributo ativo. Dessa forma, um item deixa de aparecer para novas compras sem destruir ITEM_PEDIDO histórico. A ficha técnica relaciona Prato e Ingrediente com quantidade necessária. A substituição é direcional e possui prioridade e fator de conversão. A futura disponibilidade poderá somar o consumo de todos os itens e escolher substitutos autorizados quando o ingrediente original estiver insuficiente.

12 ARQUITETURA DA APLICAÇÃO

Nós definimos uma arquitetura em camadas. O navegador contém HTML, CSS e JavaScript; a API utiliza HTTP e JSON; as Routes Flask interpretam as requisições; os Controllers validam e executam os casos de uso; e os Models mantêm os dados demonstrativos em memória. O banco relacional será conectado apenas pelo back-end no quarto bimestre.

Camada	Responsabilidade	Exemplo
View	Apresentar conteúdo e capturar eventos.	Cardápio, formulário e KDS.
JavaScript de página	Gerenciar o estado visual; futuramente chamar a API.	Carrinho, filtros e feedback.
Routes Flask	Interpretar HTTP, JSON e códigos de resposta.	POST /api/reservas.
Controllers	Validar e executar casos de uso.	Verificar conflito e criar Sessão.
Models em memória	Representar e consultar dados demonstrativos.	Reserva, Sessão, Pedido e Prato.
Banco relacional futuro	Garantir persistência e integridade no quarto bimestre.	PK, FK, unicidade e transações.

O repositório separa o front-end em módulos JavaScript e o back-end em Routes, Controllers e Models. No navegador, dados simulados sustentam as telas e os fluxos. No servidor, Blueprints expõem endpoints reais e os Controllers aplicam regras sobre Models em memória. As duas partes ainda funcionam separadamente; a integração por Fetch e a persistência ficam para a próxima etapa.

13 LÓGICA JAVASCRIPT

O JavaScript atual foi dividido por responsabilidade para evitar um arquivo monolítico e tornar explícitos os conteúdos do terceiro bimestre. `data.js` concentra arrays, objetos literais e estado compartilhado; `site.js` reúne navegação, `localStorage` e `helpers`; `components.js` contém os geradores visuais; `catalog.js` manipula o DOM do Cardápio; `cart.js` mantém o carrinho e calcula valores; `validation.js` valida Reservas; `login.js` trata o login demonstrativo; `views.js` registra somente as telas; e `a11y.js` concentra comportamentos de acessibilidade.

```
function renderCatalog(category = "Todos") {
  const grid = document.querySelector(".grid.cols-4");
  grid.innerHTML = "";
  products.forEach(product => {
    if (category !== "Todos" && product.cat !== category) return;
    const wrapper = document.createElement("div");
    wrapper.innerHTML = productCard(product).trim();
    grid.appendChild(wrapper.firstElementChild);
  });
}
```

Trecho 3 - Renderização dinâmica do Cardápio utilizando `forEach` e manipulação do DOM.

Utilizamos arrays, objetos literais, `const`, arrow functions, template literals, `find`, `filter`, `reduce`, `forEach`, `for...of` e `for...in`. O Cardápio é atualizado dinamicamente pelo DOM; o carrinho mantém itens e calcula subtotais e total em memória; os formulários possuem validações simples; e o login demonstrativo seleciona o perfil e armazena a sessão localmente. Esses elementos cobrem os conteúdos de JavaScript previstos para o terceiro bimestre.

No quarto bimestre, a modularização será ampliada com classes ES6+, encapsulamento e módulos voltados à comunicação com a API. As chamadas `Fetch` serão assíncronas, utilizarão JSON e serão tratadas com `async/await` e `try/catch`. O estado que hoje é simulado no navegador passará a refletir as respostas reais do Flask.

14 ARQUITETURA FLASK E ROTAS ESSENCIAIS

O back-end do terceiro bimestre é uma aplicação Flask executável. A factory usa `Flask(__name__)`, carrega a configuração e registra Blueprints. As Routes usam `request.get_json()` e `jsonify`; os Controllers validam os casos de uso; e os Models mantêm usuários, Bancadas, Reservas, Sessões, Pratos e Pedidos em memória. Não há banco físico, autenticação real ou integração Fetch nesta etapa.

Método e rota	Atores	Lógica essencial
GET /api	Todos	Confirma que a API está em execução.
POST/GET /api/cadastro	Cliente	Cria e consulta cadastro demonstrativo.
POST /api/login	Todos	Identifica perfil sem autenticação real.
GET /api/bancadas	Cliente/Admin	Lista Bancadas em memória.
POST/GET /api/reservas[/<id>]	Cliente/Admin	Cria e consulta Reservas em memória.
POST/GET /api/sesoes[/<id>]	Cliente/Admin	Cria e consulta Sessões em memória.
POST /api/sesoes/<id>/encerrar	Cliente/Admin	Encerra Sessão e bloqueia novos Pedidos.
GET /api/cardapio	Cliente	Retorna o Cardápio em memória.
POST/GET /api/pedido[/cadastro/<id>]	Cliente	Cria por id_sessao e consulta Pedidos.
GET /api/kds[/<id>]	Cozinheiro/Admin	Lista a fila ou retorna um Pedido com Bancada.

14.1 Exemplo atual de rota Flask

```
@reserva_bp.post("/reservas")
def reservas():
    dados = request.get_json(silent=True) or {}
    resposta, status = criar_reserva(dados)
    return jsonify(resposta), status
```

Trecho 4 - Rota Flask executável para criação de Reserva.

15 BANCO DE DADOS E DER

Nós modelamos USUARIO e suas especializações CLIENTE, COZINHEIRO e ADMINISTRADOR; BANCADA e RESERVA; SESSAO, PEDIDO e ITEM_PEDIDO; PRATO e INGREDIENTE; além de FICHA_TECNICA e SUBSTITUIDO_POR. O dicionário descreve atributos, tipos sugeridos, domínios e regras de consistência.

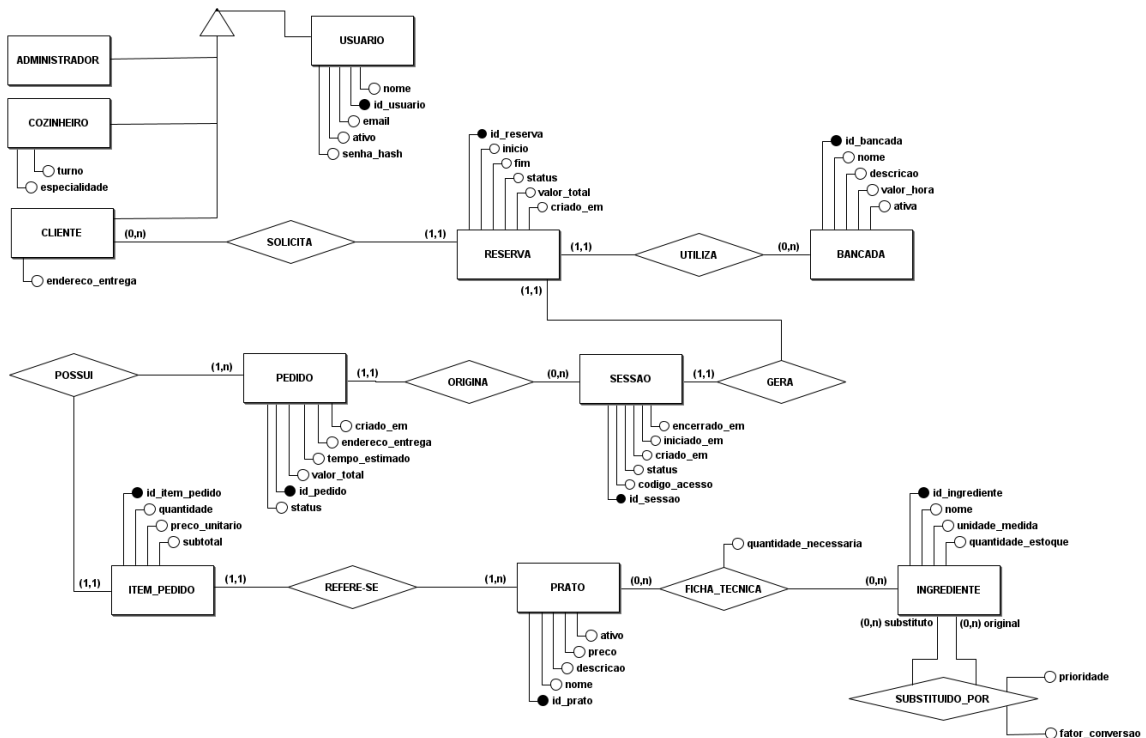


Figura 1 - Diagrama Entidade Relacionamento do Sabor e Clic. Fonte: elaboração da equipe, 2026.

A cadeia RESERVA $\leftrightarrow$ SESSAO $\rightarrow$ PEDIDO $\rightarrow$ ITEM_PEDIDO é o núcleo do modelo. CLIENTE solicita RESERVA e BANCADA é utilizada pela RESERVA. Cada item referencia um PRATO. FICHA_TECNICA resolve a relação muitos para muitos entre Prato e Ingrediente, enquanto SUBSTITUIDO_POR utiliza duas referências para Ingrediente.

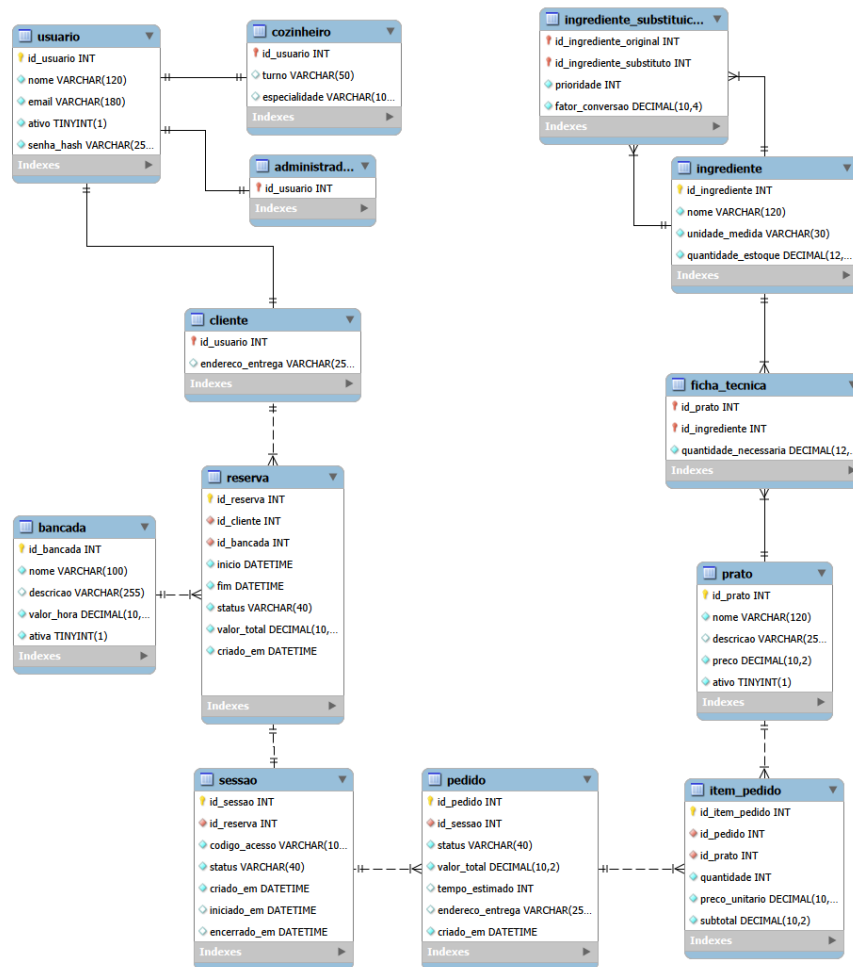


Figura 2 - Modelo de entidades e relacionamentos. Fonte: elaboração da equipe, 2026.

Definimos regras de integridade: e-mail único, valores não negativos, fim posterior ao início, Sessão única por Reserva, quantidade positiva, Pedido com itens e chaves estrangeiras obrigatórias. No quarto bimestre, o modelo físico deverá traduzir essas regras para SQL, constraints, índices, inserções básicas, consultas e triggers de estoque.

16 SEGURANÇA E NÍVEIS DE ACESSO

Definimos RBAC com três papéis. Cliente acessa seus próprios dados; Cozinheiro opera Pedidos e KDS; Administrador gerencia cadastros e visão geral. No protótipo, essa divisão aparece nos menus. Na aplicação final, ela será aplicada no servidor por decorators ou middleware, porque ocultar um botão não impede uma chamada HTTP direta.

Risco	Decisão arquitetural
Senha exposta	Armazenar hash forte com salt; nunca senha em texto ou cifra reversível.
Preço manipulado	Recalcular total no Flask a partir do catálogo.
Reserva duplicada	Validar sobreposição dentro de transação.
SQL Injection	Utilizar ORM ou consultas parametrizadas.
XSS	Escapar dados e preferir <code>textContent</code> para conteúdo externo.
CSRF/cookie	Configurar <code>HttpOnly</code> , <code>Secure</code> , <code>SameSite</code> e proteção CSRF quando aplicável.
Acesso indevido	Verificar identidade, propriedade do recurso e papel em cada rota.

Removemos o utilitário que apenas concatenava uma chave ao texto, pois ele não implementava criptografia. Nenhuma senha é armazenada pelo backend demonstrativo. No quarto bimestre, a autenticação real deverá usar hash unidirecional com salt, mensagens genéricas para falhas de login e logs sem informações sensíveis.

17 IDENTIDADE VISUAL E PROTOTIPAÇÃO

Nós construímos uma identidade inspirada em materiais e cores gastronômicas. Utilizamos grafite #2B241E para estrutura, fundo #F6F1E8, cobre #BB6B2F como destaque, verde #4B7A52 para sucesso, açafrão #B9832A para atenção e páprica #A6432E para erro. Archivo aparece em títulos e Hanken Grotesk nos elementos de interface.

Criamos componentes consistentes: menu lateral, cartões, badges, botões, tabelas, filtros, indicadores, timeline e colunas do KDS. CSS Grid e Flexbox organizam o conteúdo, enquanto media queries adaptam a interface. As imagens das Bancadas e dos produtos aproximam o protótipo de uma experiência de uso real.

O [protótipo navegável do Sabor e Clic no Figma](#) reúne os wireframes e as telas usadas como referência. O front-end atual demonstra as áreas de Cliente, Cozinheiro e Administrador.

18 REQUISITOS FUNCIONAIS E NÃO FUNCIONAIS

Código	Requisito funcional	Estágio
RF01	Entrar e acessar interface correspondente ao papel.	Demonstrado no protótipo; autenticação real é próxima etapa.
RF02	Consultar e reservar Bancadas.	Interface concluída; API cria e consulta Reservas em memória.
RF03	Criar, ativar e encerrar Sessão.	Interface e endpoints de criação, consulta e encerramento concluídos em memória.
RF04	Consultar Cardápio e manter carrinho.	Implementado em memória.
RF05	Criar e acompanhar Pedido.	Interface e API em memória concluídas; integração Fetch planejada.
RF06	Operar fila no KDS.	Layout e consulta JSON concluídos; transições persistentes planejadas.
RF07	Administrar catálogo, Ingredientes e usuários.	Telas demonstrativas concluídas; CRUD planejado.

Requisito não funcional	Como tratamos
Usabilidade	Hierarquia, feedback, navegação por perfil e consistência visual.
Responsividade	Grid, Flexbox e media queries no CSS.
Manutenibilidade	Separação real entre Routes, Controllers e Models, além dos módulos JavaScript por responsabilidade.
Segurança	Perfis e validações de entrada demonstrados; autenticação, cookies e RBAC previstos.
Integridade	Validações em memória nesta etapa; PK, FK, unicidade e transações previstas no banco.
Desempenho	Cálculos e consultas em memória; índices e filtros no servidor ficam para a persistência.

19 TECNOLOGIAS UTILIZADAS

Tecnologia	Aplicação no projeto
HTML5	Estrutura das páginas e formulários.
CSS3	Identidade visual, componentes e responsividade.
JavaScript ES6+	DOM, estado, arrays, templates, carrinho e cálculos.
JSON e Fetch API	JSON implementado na API Flask; Fetch permanece planejado para o quarto bimestre.
Python e Flask	API executável, validações e dados demonstrativos em memória.
Banco relacional/MySQL Workbench	DER, MER, EER e futura persistência SQL.
Git e GitHub	Versionamento e integração do trabalho da equipe.

20 O QUE CONCLUÍMOS E PRÓXIMAS ETAPAS

20.1 O que concluímos

Concluimos o protótipo navegável, a identidade visual, as telas dos três atores, o Cardápio, o carrinho em memória, a visualização de Pedidos, o layout do KDS, o DER, o MER e o dicionário de dados. Também modularizamos o JavaScript para evidenciar os conteúdos da etapa e implementamos uma API Flask executável com Blueprints, Controllers, Models em memória e respostas JSON.

20.2 O que iniciamos

Implementamos a factory Flask e endpoints reais para cadastro/login demonstrativos, Bancadas, Reservas, Sessões, Cardápio, Pedidos e KDS. O Pedido usa id_sessão; o servidor valida a Sessão ativa, recalcula valores e mantém o fluxo Reserva → Sessão → Pedido → KDS. Os dados são reiniciados com o servidor. Autenticação, banco físico e Fetch permanecem para a próxima etapa.

20.3 Próximas etapas

Nas próximas etapas, substituiremos os Models em memória por persistência relacional, implementaremos autenticação e RBAC, conectaremos o front-end à API por Fetch/JSON e persistiremos as transições do KDS. Também serão acrescentados POO completa em JavaScript, tratamento de loading/erros, constraints, consultas, triggers e testes de integração.

21 ATENDIMENTO AOS REQUISITOS DO TERCEIRO BIMESTRE

Requisito	Situação	Evidência produzida
DER	Concluído	DER_saborclic.png, MER_saborclic.png e EER_saborclic.mwb.
Dicionário de dados	Concluído	Documento em DOCX e PDF com entidades, atributos e relações.
Wireframes/Figma	Concluído	Link público do Figma e protótipo navegável registrados.
Identidade visual	Concluída	Paleta, tipografia, marca e componentes em CSS.
Cardápio Digital	Concluído como protótipo	Produtos, categorias, imagens, disponibilidade e carrinho.
KDS	Concluído como protótipo	Colunas, tickets, tempos e ações simuladas.
JavaScript/DOM	Concluído para a etapa	validation.js e login.js validam no navegador; Controllers validam Reservas, Sessões e Pedidos no Flask.
Validações	Concluído para a etapa	Factory Flask, Blueprints, Routes, Controllers, Models em memória, JSON e fluxo Reserva → Sessão → Pedido → KDS.
Flask/MVC/rotas/JSON	Definido e demonstrado	Perfis e separação visual existem; autenticação real, cookies e RBAC serão aplicados no quarto bimestre.
Segurança e RBAC	Definido e demonstrado no protótipo	Perfis e separação visual existem; autenticação e RBAC de autoridade serão aplicados no servidor.

22 ESTADO ATUAL DA IMPLEMENTAÇÃO E INTEGRAÇÃO

Esta seção consolida a implementação atual. No terceiro bimestre, o front-end utiliza dados simulados para demonstrar as telas, enquanto o Flask expõe uma API JSON executável com dados separados em memória. No quarto bimestre, Fetch conectará essas partes e o banco substituirá a memória como fonte dos dados.

22.1 Front-end e JavaScript

O front-end possui áreas separadas para Cliente, Cozinheiro e Administrador. O JavaScript está dividido entre data.js, site.js, components.js, catalog.js, cart.js, validation.js, login.js, views.js e a11y.js, cobrindo arrays, objetos, for...of, for...in, forEach, DOM, validação e cálculos em memória.

22.2 Back-end Flask em memória

A factory Flask registra Blueprints para cadastro/login demonstrativos, Bancadas/Reservas, Sessões, Cardápio/Pedidos e KDS. As Routes recebem JSON, os Controllers validam os casos de uso e os Models mantêm dados simulados. Pedido referencia id_sessao, preservando a cadeia até a Reserva e a Bancada. Não há persistência ou autenticação real.

22.3 Integração e próxima etapa

O fluxo permanece Reserva → Sessão → Cardápio/Carrinho → Pedido → KDS. A API já recebe identificadores e quantidades, valida a Sessão, recalcula valores e devolve JSON quando chamada diretamente. Na integração do quarto bimestre, o JavaScript fará essas chamadas por Fetch, o banco persistirá os dados e o KDS enviará alterações de estado ao backend.

REFERÊNCIAS

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO. Projeto Integrador Sabor & Clic: sistema de gestão para dark kitchens e espaços gastronômicos. Versão de 31 de agosto de 2026. São Paulo: IFSP, 2026.

SABOR E CLIC. Repositório Sabor-Clic. GitHub, 2026. Disponível em: <https://github.com/FaellEmiliano/Sabor-Clic>.

SABOR E CLIC. Dicionário de Dados Sabor e Clic. São Paulo, 2026.